SQL Add-On Exercise: Solutions

Introduction to SQL for Beginners, `simple_orders_dataset`

Queries run on MySQL 8.0 against the 100-row `simple_orders.csv`

Setup

The table `simple_orders_dataset` has seven columns: `order_id`, `customer_name`, `city`, `product`, `quantity`, `price` and `order_date`. It holds 100 orders, with no missing values.

- In MySQL Workbench, create a schema, then right-click Tables and choose *Table Data Import Wizard*.
- Select the CSV and name the new table `simple_orders_dataset`. The file is called `simple_orders.csv`, so the table name must be typed in.
- Run `SELECT COUNT(*) FROM simple_orders_dataset;` to confirm 100 rows were loaded.

Note on the data: the cities are Abuja, Ibadan, Kano, Lagos and Port Harcourt, and the products are Chair, Desk, Laptop, Monitor and Mouse. There is no Enugu and no Tablet, which affects Tasks 3 and 6.

Task 1

Task: Retrieve all the records from the table.

Query:

```
SELECT * FROM simple_orders_dataset;
```

What it does in plain English: Shows every column of every order in the table. The star (`*`) means "all columns", and with no `WHERE` clause nothing is filtered out, so all 100 orders come back.

MySQL output:

```
+----------+-----------------------+---------------+---------+----------+-----------+------------+
| order_id | customer_name         | city          | product | quantity | price     | order_date |
+----------+-----------------------+---------------+---------+----------+-----------+------------+
| ORD1001  | Norma Fisher          | Kano          | Monitor |        1 |  92103.03 | 2024-02-20 |
| ORD1002  | Katelyn Hull          | Ibadan        | Monitor |        4 | 321545.55 | 2024-06-10 |
| ORD1003  | John Silva            | Port Harcourt | Monitor |        3 | 205016.95 | 2024-06-25 |
| ORD1004  | Charles Taylor        | Abuja         | Chair   |        2 | 100079.57 | 2024-01-19 |
| ORD1005  | Danielle Browning     | Lagos         | Chair   |        3 | 345566.20 | 2024-05-16 |
| ORD1006  | Stephanie Collins     | Ibadan        | Chair   |        2 | 109931.35 | 2024-05-21 |
| ORD1007  | Samuel Rivera         | Lagos         | Mouse   |        4 | 196815.16 | 2024-04-18 |
| ORD1008  | Jeffrey Carr          | Port Harcourt | Monitor |        3 | 214588.67 | 2024-06-26 |
+----------+-----------------------+---------------+---------+----------+-----------+------------+

100 rows in set (0.00 sec)
```

Showing first 8 of 100 rows in this document. MySQL returned all 100.

Task 2

Task: Display only the customer_name and city columns.

Query:

```
SELECT customer_name, city FROM simple_orders_dataset;
```

What it does in plain English: Instead of all columns, we list just the two we want, separated by a comma. MySQL returns who ordered and where they are, for all 100 orders.

MySQL output:

```
+------------------------+---------------+
| customer_name          | city          |
+------------------------+---------------+
| Norma Fisher           | Kano          |
| Katelyn Hull           | Ibadan        |
| John Silva             | Port Harcourt |
| Charles Taylor         | Abuja         |
| Danielle Browning      | Lagos         |
| Stephanie Collins      | Ibadan        |
| Samuel Rivera          | Lagos         |
| Jeffrey Carr           | Port Harcourt |
+------------------------+---------------+

100 rows in set (0.00 sec)
```

Showing first 8 of 100 rows in this document. MySQL returned all 100.

Task 3

Task: Get all orders made in the city of 'Enugu'.

Query:

```
SELECT * FROM simple_orders_dataset WHERE city = 'Enugu';
```

What it does in plain English: `WHERE` keeps only rows whose city is exactly Enugu. MySQL returned an empty set because no order in this dataset has that city. The cities present are Abuja, Ibadan, Kano, Lagos and Port Harcourt.

MySQL output:

```
Empty set (0.00 sec)
```

Task 4

Task: Retrieve orders where the quantity is more than 2.

Query:

```
SELECT * FROM simple_orders_dataset WHERE quantity > 2;
```

What it does in plain English: Keeps only orders with a quantity above 2, that is 3, 4 or 5 items. Orders of exactly 2 are not included because the test is "greater than", not "greater than or equal to". 59 orders qualify.

MySQL output:

```
+----------+-----------------------+---------------+---------+----------+-----------+------------+
| order_id | customer_name         | city          | product | quantity | price     | order_date |
+----------+-----------------------+---------------+---------+----------+-----------+------------+
| ORD1002  | Katelyn Hull          | Ibadan        | Monitor |        4 | 321545.55 | 2024-06-10 |
| ORD1003  | John Silva            | Port Harcourt | Monitor |        3 | 205016.95 | 2024-06-25 |
| ORD1005  | Danielle Browning     | Lagos         | Chair   |        3 | 345566.20 | 2024-05-16 |
| ORD1007  | Samuel Rivera         | Lagos         | Mouse   |        4 | 196815.16 | 2024-04-18 |
| ORD1008  | Jeffrey Carr          | Port Harcourt | Monitor |        3 | 214588.67 | 2024-06-26 |
| ORD1009  | Juan Ramos            | Abuja         | Chair   |        4 | 156057.29 | 2024-04-11 |
| ORD1012  | April Snyder          | Kano          | Laptop  |        5 | 173765.10 | 2024-06-17 |
| ORD1013  | Nicholas Padilla      | Port Harcourt | Desk    |        3 | 246867.61 | 2024-01-13 |
+----------+-----------------------+---------------+---------+----------+-----------+------------+

59 rows in set (0.00 sec)
```

Showing first 8 of 59 rows in this document. MySQL returned all 59.

Task 5

Task: Display the top 5 highest priced orders.

Query:

```
SELECT * FROM simple_orders_dataset ORDER BY price DESC LIMIT 5;
```

What it does in plain English: `ORDER BY price DESC` sorts from the highest price down, and `LIMIT 5` keeps only the first five rows. The most expensive order is ORD1098 (Meghan Barron, Desk) at 346064.08.

MySQL output:

```
+----------+------------------+---------------+---------+----------+-----------+------------+
| order_id | customer_name    | city          | product | quantity | price     | order_date |
+----------+------------------+---------------+---------+----------+-----------+------------+
| ORD1098  | Meghan Barron    | Ibadan        | Desk    |        1 | 346064.08 | 2024-06-10 |
| ORD1005  | Danielle Browning | Lagos         | Chair   |        3 | 345566.20 | 2024-05-16 |
| ORD1072  | Jacob Bradley    | Port Harcourt | Chair   |        3 | 331382.89 | 2024-06-13 |
| ORD1037  | Daniel Owen      | Kano          | Desk    |        1 | 328065.27 | 2024-05-11 |
| ORD1002  | Katelyn Hull     | Ibadan        | Monitor |        4 | 321545.55 | 2024-06-10 |
+----------+------------------+---------------+---------+----------+-----------+------------+

5 rows in set (0.00 sec)
```

Task 6

Task: List all customers who ordered a 'Tablet' product.

Query:

```
SELECT customer_name FROM simple_orders_dataset WHERE product = 'Tablet';
```

What it does in plain English: Returns the names of customers whose product is exactly Tablet. MySQL returned an empty set because this dataset has no Tablet. The products present are Chair, Desk, Laptop, Monitor and Mouse.

MySQL output:

```
Empty set (0.00 sec)
```

Task 7

Task: Use a CASE statement to categorize orders based on quantity (1 = Single, 2–3 = Small, more than 3 = Bulk).

Query:

```
SELECT customer_name, quantity,
CASE
  WHEN quantity = 1 THEN 'Single'
  WHEN quantity BETWEEN 2 AND 3 THEN 'Small'
  ELSE 'Bulk'
END AS order_size
FROM simple_orders_dataset;
```

What it does in plain English: `CASE` works like an if/else chain, checked top to bottom: quantity 1 gives Single, quantity 2 to 3 gives Small, and everything else (4 or 5) gives Bulk. The result is shown in a new column called `order_size`. Across all 100 orders this gives 25 Single, 37 Small and 38 Bulk.

MySQL output:

```
+-----------------------+----------+------------+
| customer_name         | quantity | order_size |
+-----------------------+----------+------------+
| Norma Fisher          |        1 | Single     |
| Katelyn Hull          |        4 | Bulk       |
| John Silva            |        3 | Small      |
| Charles Taylor        |        2 | Small      |
| Danielle Browning     |        3 | Small      |
| Stephanie Collins     |        2 | Small      |
| Samuel Rivera         |        4 | Bulk       |
| Jeffrey Carr          |        3 | Small      |
+-----------------------+----------+------------+
```

```
100 rows in set (0.01 sec)
```

Showing first 8 of 100 rows in this document. MySQL returned all 100.

Task 8

Task: Sort all orders from Port Harcourt by price (highest first).

Query:

```
SELECT * FROM simple_orders_dataset WHERE city = 'Port Harcourt' ORDER BY price DESC;
```

What it does in plain English: First `WHERE` keeps only Port Harcourt orders, then `ORDER BY price DESC` puts the most expensive first. There are 21 Port Harcourt orders, and the top one is ORD1072 (Jacob Bradley, Chair) at 331382.89.

MySQL output:

```
+----------+------------------+---------------+---------+----------+-----------+------------+
| order_id | customer_name    | city          | product | quantity | price     | order_date |
+----------+------------------+---------------+---------+----------+-----------+------------+
| ORD1072  | Jacob Bradley    | Port Harcourt | Chair   |        3 | 331382.89 | 2024-06-13 |
| ORD1053  | Angela Wheeler   | Port Harcourt | Monitor |        4 | 290458.16 | 2024-05-15 |
| ORD1013  | Nicholas Padilla | Port Harcourt | Desk    |        3 | 246867.61 | 2024-01-13 |
| ORD1027  | Matthew Davies   | Port Harcourt | Laptop  |        3 | 215248.52 | 2024-06-27 |
| ORD1008  | Jeffrey Carr     | Port Harcourt | Monitor |        3 | 214588.67 | 2024-06-26 |
| ORD1019  | Jill Carlson     | Port Harcourt | Monitor |        1 | 209507.83 | 2024-03-15 |
| ORD1092  | Scott Miles      | Port Harcourt | Desk    |        3 | 205659.34 | 2024-06-24 |
| ORD1003  | John Silva       | Port Harcourt | Monitor |        3 | 205016.95 | 2024-06-25 |
+----------+------------------+---------------+---------+----------+-----------+------------+

21 rows in set (0.00 sec)
```

Showing first 8 of 21 rows in this document. MySQL returned all 21.

Task 9

Task: Show only the unique product types sold in the dataset.

Query:

```
SELECT DISTINCT product FROM simple_orders_dataset;
```

What it does in plain English: `DISTINCT` removes repeated values, so each product name appears once. There are five product types: Monitor, Chair, Mouse, Laptop and Desk.

MySQL output:

```
+---------+
| product |
+---------+
| Monitor |
| Chair   |
| Mouse   |
| Laptop  |
| Desk    |
+---------+

5 rows in set (0.00 sec)
```

Task 10

Task: Final Challenge: Lagos orders for 'Laptop', with price and a Low / Medium / High tier (below 50,000 = Low; 50,000 to 150,000 = Medium; above 150,000 = High).

Query:

```
SELECT customer_name, price,
CASE
  WHEN price < 50000 THEN 'Low'
  WHEN price BETWEEN 50000 AND 150000 THEN 'Medium'
  ELSE 'High'
END AS price_tier
FROM simple_orders_dataset
WHERE city = 'Lagos' AND product = 'Laptop';
```

What it does in plain English: `WHERE` with `AND` keeps only rows that are both in Lagos and for a Laptop. `CASE` then labels each price. There are four such orders: two are High (299652.27 and 159160.90) and two are Medium (68923.72 and 70918.02). No Lagos laptop costs less than 50,000, so there are no Low rows.

MySQL output:

```
+-----------------+-----------+------------+
| customer_name   | price     | price_tier |
+-----------------+-----------+------------+
| Robert Garcia   | 299652.27 | High       |
| David Walsh     |  68923.72 | Medium     |
| Adam Long       |  70918.02 | Medium     |
| Daniel Williams | 159160.90 | High       |
+-----------------+-----------+------------+

4 rows in set (0.00 sec)
```

Portfolio checklist

- Run each query in your own MySQL Workbench and take a screenshot showing the query and its result grid.
- Save all ten queries in one `.sql` file.
- For Tasks 3 and 6, add a short note that the empty result is correct because Enugu and Tablet are not in the data.